AI Assisted Coding
Lab Assignment - 10.1

Name : Shivanand Rama
Hall.t.No. : 2303A51037
Batch No. : 01

Lab 10 – Code Review and Quality: Using AI to Improve Code Quality and Readability
Lab Objectives

- Use AI for automated code review and quality enhancement.
- Identify and fix syntax, logical, performance, and security issues in Python code.
- Improve readability and maintainability through structured refactoring and comments.
- Apply prompt engineering for targeted improvements.
- Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices

Lab Outcomes

1. Students will be able to use AI tools to review code.
2. Students will be able to improve code quality and readability.
3. Students will be able to identify and fix common coding issues.

Task Description #1 – Syntax and Logic Errors
Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.
Sample Input Code:

```
# Calculate average score of a student
def calc_average(marks):
    total = 0
```

```
        for m in marks:
            total += m
            average = total / len(marks)
        return average # Typo here
    marks = [85, 90, 78, 92]
    print("Average Score is ", calc_average(marks)
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Given code in task :

```python
task_1.py > ...
# Use AI to identify and fix syntax and logic errors in a faulty Python script.
# Sample Input Code:
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
        average = total / len(marks)
    return average # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)
```

Copilot suggested code :

```python
# Corrected Code with Explanations:
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)  # Moved this line outside the loop to calculate average after summing all marks
    return average  # Fixed typo from 'avrage' to 'average'
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))  # Added missing closing parenthesis at the end of the print statement
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-10.1/task
_1.py
Average Score is  86.25
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1>
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

Given code in task :

```
task_2.py > ...
# Use AI to refactor Python code to follow PEP 8 style guidelines.
# Sample Input Code:
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
```

Copilot suggested code :

```
# Refactored Code with Explanations:
def area_of_rectangle(length, breadth):
    """Calculate the area of a rectangle given its length and breadth."""
    return length * breadth  # Added spaces around operators and renamed variables for clarity

print(area_of_rectangle(10, 20))  # Updated function name and added spaces for better readability
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted
_Coding/lab_assignment-10.1/task_2.py
200
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1>
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
    return x*y/100
a=200
b=15
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

Given code in task :

```
task_3.py > ...
# Use AI to make code more readable without changing its logic.
# Sample Input Code:
def calculate_percentage(value, percentage):
        return value*percentage/100
amount = 200
percent = 15
print(calculate_percentage(amount, percent))
```

Copilot suggested code :

```
# Readability Enhanced Code with Explanations:
def calculate_percentage(value, percentage):
    """Calculate the percentage of a given value."""
    return value * percentage / 100  # Added spaces around operators for better readability
amount = 200  # Renamed variable 'a' to 'amount' for clarity
percent = 15  # Renamed variable 'b' to 'percent' for clarity
print(calculate_percentage(amount, percent))  # Updated function name and added spaces for better readability
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-10
.1/task_3.py
30.0
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1>
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Given code in task :

```
task_4.py > ...
# Use AI to break repetitive or long code into reusable functions.
# Sample Input Code:
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Copilot suggested code :

```
# Refactored Code with Reusable Function
def welcome_students(students):
    for student in students:
        print("Welcome", student)
# Sample Input
students = ["Alice", "Bob", "Charlie"]
# Call the function to welcome students
welcome_students(students)
# In this refactored code, we have created a function called `welcome_students` that takes a list of students as an argument.
# The function iterates through the list and prints a welcome message for each student.
# This approach eliminates the need for repetitive print statements and makes the code more maintainable and reusable.
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-10
.1/task_4.py
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1>
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

Given code in task :

```python
# Use AI to make the code run faster.
# Sample Input Code:
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
```

Copilot suggested code :

```python
# Optimized code using list comprehensions
nums = [i for i in range(1, 1000000)]
squares = [n**2 for n in nums]
print(len(squares))

# Alternatively, using NumPy for vectorized operations
import numpy as np
nums = np.arange(1, 1000000)
squares = nums ** 2
print(len(squares))
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted
_Coding/lab_assignment-10.1/task_5.py
999999
999999
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1>
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:                    return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Given code in task :

task_6.py > ...

```python
# Use AI to simplify overly complex logic.
# Sample Input Code:
def grade(score):
    if score ≥ 90:
        return "A"
    else:
        if score ≥ 80:
            return "B"
        else:
            if score ≥ 70:
                return "C"
            else:
                if score ≥ 60:
                    return "D"
                else:
                    return "F"
```

Copilot suggested code :

```python
# Optimized code using elif statements
def grade(score):
    if score ≥ 90:
        return "A"
    elif score ≥ 80:
        return "B"
    elif score ≥ 70:
        return "C"
    elif score ≥ 60:
        return "D"
    else:
        return "F"
# Alternatively, using a dictionary mapping
def grade(score):
    grade_mapping = {
        range(90, 101): "A",
        range(80, 90): "B",
        range(70, 80): "C",
        range(60, 70): "D",
        range(0, 60): "F"
    }
    for score_range, grade in grade_mapping.items():
        if score in score_range:
            return grade
# Example usage
print(grade(95))  # Output: A
print(grade(85))  # Output: B
print(grade(75))  # Output: C
print(grade(65))  # Output: D
print(grade(55))  # Output: F
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-10
.1/task_6.py
A
B
C
D
F
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-10.1>
```